

ИДЗ. ТЕСТУВАННЯ ТА АВТОМАТИЗАЦІЯ

Мета: Навчитися розробляти автоматизовані тести для програмних продуктів, використовувати інструменти оцінки покриття коду тестами та налаштовувати процеси безперервної інтеграції (CI) за допомогою GitHub Actions.

ЧАСТИНА 1: Базовий рівень (оцінюється у 4 бали)

Завдання 1.1 "Основи Unit-тестування"

Контекст: Ви отримали від колеги модуль валідації паролів для нової системи реєстрації:

```
# файл security_utils.py
def validate_password(password):
    if len(password) < 8:
        return False

    if not any(char.isdigit() for char in password):
        return False

    if "admin" in password.lower():
        return False

    return True
```

Задача: написати Unit-тести для модуля реєстрації паролів.

- 1) Встановіть бібліотеку для тестування: `pip install pytest`.
- 2) Створіть файл `test_security.py`.
- 3) Використовуючи бібліотеку `pytest`, напишіть набір тестів, які перевірять функцію `validate_password` на наступні сценарії (кожен сценарій — окрема тестова функція):
 - Позитивний сценарій: Пароль відповідає всім вимогам (наприклад, `Super_User_2026`).
 - Критерій довжини: Пароль занадто короткий (менше 8 символів).
 - Критерій цифр: Пароль містить лише літери.
 - Критерій заборонених слів: Пароль містить слово `admin` у різних варіантах написання (`Admin12345`, `root_admin_pass`).
- 4) Запустіть тести через консоль командою `pytest test_security.py` та переконайтеся, що всі вони пройшли успішно (статус `PASSED`).

Завдання 1.2 "Автоматизація та CI/CD (GitHub Actions)"

Контекст: У професійній розробці тести запускаються автоматично при кожному оновленні коду. Цей функціонал є частиною CI/CD пайплайну і може бути реалізований через GitHub Actions, який є де-факто галузевим стандартом для сучасного CI/CD конвеєра в Python-проектах.

Задача: Реалізувати CI/CD конвеєр з автоперевіркою розроблених в завданні 1.1 тестів за допомогою GitHub Actions.

- 1) Ініціалізація проєкту, який містить security_utils.py та test_security.py:
`git init`
- 2) Створення та налаштування .gitignore:

```
# файл .gitignore
__pycache__/
.pytest_cache/
.coverage
htmlcov/
venv/
.env
*.pyc
```
- 3) У корені проєкту створіть папку .github, а в ній — папку workflows. У папці workflows створіть файл tests.yml та скопіюйте туди наступний ГОТОВИЙ шаблон:

```
name: Python Tests
on: [push]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          pip install pytest pytest-cov
      - name: Run tests with coverage
        run: |
          pytest --cov=security_utils test_security.py
```
- 4) Створіть порожній репозиторій на GitHub
- 5) Відправте код проєкту у GitHub:

```
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin [URL_вашого_репозиторію]
git push -u origin main
```
- 6) Перейдіть на вкладку Actions у вашому репозиторії. Переконайтеся, що процес завершився успішно та відсоток покриття коду тестами.

ЧАСТИНА 2 (середній рівень): оцінюється у 3 бали.

Завдання 2.1 "Тестування асинхронного пошукового агента"

Контекст: Ваша корутина `recursive_search` із Лабораторної роботи №6 (завдання 1.2) має працювати бездоганно, оскільки помилка в системному сканері може призвести до пропуску важливих даних. Тестувати таку функцію на "живих" папках небезпечно та незручно, тому доречним буде створення ізолюваного середовища.

Задача: Напишіть асинхронний тест, який перевіряє, чи правильно функція

знаходить файли за розширенням у заданій структурі папок.

Вимоги до реалізації:

- 1) *Бібліотеки*: Вам знадобляться `pytest` та `pytest-asyncio`.
- 2) *Фікстура `tmp_path`*: Використовуйте цей аргумент у вашому тесті. Він дозволяє створювати папки та файли за допомогою методів `makedirs()` та `write_text()`.
- 3) *Декоратор*: Оскільки ми тестуємо корутину, функція тесту повинна бути помічена `@pytest.mark.asyncio` та бути асинхронною (`async def`).

Завдання 2.2 "Тестування механізмів переривання та сигналів"

Контекст: У системному програмуванні критично важливо, щоб довгі операції (як-от сканування диска) могли бути перервані миттєво. Ваша функція `recursive_search` з ЛР №6 використовує об'єкт `asyncio.Event` для зупинки. Ви маєте перевірити, чи дійсно цей механізм працює коректно.

Задача: Написати асинхронний Unit-тест, який перевіряє "слухняність" пошукового агента. Тест має підтвердити, що якщо прапорець зупинки піднято, функція припиняє роботу, навіть якщо вона ще не встигла перевірити всі файли.

Вимоги до реалізації:

- 1) *Механіка тесту*:
 - Створіть тимчасову структуру з 10-20 файлів (за допомогою `tmp_path`).
 - Створіть об'єкт `stop_event = asyncio.Event()`.
 - Встановіть сигнал зупинки ще до початку пошуку: `stop_event.set()`.
- 2) *Дія*: Викличте `recursive_search` у цьому стані.
- 3) *Очікуваний результат (Assert)*: Список знайдених файлів (`found_files`) повинен залишитися порожнім, попри те, що файли в папці існують. Це доведе, що функція перевіряє `stop_event` на самому початку кожної ітерації.

ЧАСТИНА 3 (високий рівень): оцінюється у 3 бали. Дозволяється обрати на вибір дві з 3-х запропонованих завдань.

Завдання 3.1 Тестування відмовостійкості та обробки винятків (Mocking & Error Handling)

Контекст: У реальних розподілених системах мережа нестабільна. Ваш код повинен не просто отримувати дані, а й коректно реагувати на "погану" поведінку сервера: таймаути, обриви з'єднання та некоректні формати даних.

Задача: За допомогою бібліотеки `pytest-respx` або `unittest.mock` створіть комплексний тестовий сценарій для функції `check_resource` (з ЛР №6), який імітує такі критичні ситуації:

- 1) *Таймаут з'єднання*: Сервер не відповідає вчасно. Тест повинен перевірити, що ваша функція не висить вічно, а коректно викидає або обробляє виняток таймууту.
- 2) *Ланцюжок невдач (Side Effects)*: Імітуйте ситуацію, коли перші два виклики функції завершуються помилкою 503 (**Service Unavailable**), а третій успішно. Це дозволить перевірити логіку повторних спроб (якщо студент її реалізував) чи стабільність програми за часткових збоїв.
- 3) *Пошкоджені дані*: Сервер повертає статус 200, але замість очікуваного JSON надсилає порожній файл або некоректний HTML-код. Тест повинен підтвердити, що парсер даних у вашій функції не викликає падіння всієї програми.

Завдання 3.2 Тестування наявності станів гонитви (Race Conditions & Atomic Updates)

Основа: Завдання 3.1 з ЛР №6 (використання черг та спільних ресурсів).

Контекст: В асинхронному програмуванні часто виникає помилкова ілюзія, що код виконується послідовно і "безпечно". Проте, коли кілька корутин одночасно працюють зі спільним ресурсом (наприклад, лічильником оброблених файлів або базою даних), можуть виникати стани гонитви (Race Conditions).

Задача: Створіть тест, який перевіряє коректність оновлення спільного ресурсу декількома паралельними воркерами.

- 1) *Доопрацювання коду*: Створіть клас **ResourceCounter** з методом **increment()**. Навмисно додайте в метод невелику асинхронну затримку (**await asyncio.sleep(0)**) між зчитуванням значення та його записом, щоб спровокувати потенційну помилку.
- 2) *Підготовка тесту (Arrange)*:
 - Запустіть 100 "воркерів" (корутин), кожен з яких має 10 разів викликати **increment()**.
- 3) *Дія (Act)*: Виконайте всі воркери через **asyncio.gather()**.
- 4) *Перевірка (Assert)*:
 - Тест має перевірити, чи дорівнює фінальне значення лічильника рівно 1000.
 - Якщо через відсутність синхронізації (наприклад, **asyncio.Lock**) значення менше — тест має виявити баг.
- 5) *Друга частина завдання*: Виправте код за допомогою **asyncio.Lock** та підтвердіть тестом, що тепер система працює стабільно.

Завдання 3.3 Тестування надійності обміну даними (Producer-Consumer Queue)

Основа: Завдання 3.1 з ЛР №6 (черга повідомлень та маркери завершення).

Контекст: У ЛР №6 було реалізовано систему, де **Producer** надсилає дані, а **Consumer** їх обробляє. Найкритичніший момент — зупинка системи. Тест має підтвердити, що ваш механізм "маркерів завершення" (наприклад, надсилання **None** у чергу) дозволяє **Consumer**-у вичитати всі дані до того, як він завершить роботу..

Задача: Напишіть інтеграційний тест для перевірки повноти обробки даних при завершенні роботи.

1) *Підготовка (Arrange):*

- Створіть об'єкт `asyncio.Queue()`.
- Використовуйте спрощені версії ваших **Producer** та **Consumer** із ЛР №6.
- **Producer** має покласти в чергу 10 повідомлень і одразу після них — маркер завершення (**None**).

2) *Дія (Act):*

- Запустіть **Consumer** як задачу (`asyncio.create_task`).
- Дочекайтеся повного завершення роботи **Consumer**-а (коли він вийде з циклу, отримавши **None**).

3) *Перевірка (Assert):*

- Створіть лічильник всередині **Consumer**-а або передайте список для результатів.
- Тест має підтвердити, що кількість оброблених повідомлень рівно 10.
- Якщо **Consumer** завершився раніше, ніж опрацював усі повідомлення з черги — тест має впасти.

КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке Unit-тестування і яка його головна мета в розробці ПЗ?
2. Що таке CI/CD і навіщо автоматизувати запуск тестів при кожному git push?
3. Навіщо потрібен файл .gitignore у проєкті з тестами та віртуальним середовищем?
4. Що показує метрика "Code Coverage" (покриття коду) і чому вона важлива?
5. У чому особливість тестування асинхронних функцій (`async def`) порівняно зі звичайними?
6. Для чого в тестах використовують механізм Mocking (імітацію відповідей сервера)?
7. Що таке "фікстури" (fixtures) у pytest і яка їхня роль у створенні тестового середовища?